

Quy hoạch tuyến tính và thuật toán đơn hình trong thi tin học

Li Yuqian

2007

Nguồn gốc: Linear programming in informatics contests: concise and efficient simplex implementation and applications

IOI China candidate team papers / 2007 / day 1 / Li Yuqian.ppt

Tóm tắt

Bản trình chiếu giới thiệu quy hoạch tuyến tính từ góc nhìn thi lập trình: khi một bài toán có mục tiêu tuyến tính và các ràng buộc tuyến tính, ta có thể mô hình hóa nó thành một hệ tối ưu liên tục. Trọng tâm của bài là cách dùng thuật toán đơn hình để giải những mô hình ấy bằng một chương trình tương đối ngắn, đồng thời nhấn mạnh rằng trong thực nghiệm đơn hình thường nhanh hơn nỗi lo ban đầu của thí sinh.

Các slide đi theo ba mạch: định nghĩa và dạng chuẩn của quy hoạch tuyến tính; một vài mô hình ứng dụng, đặc biệt là luồng đa hàng hóa; và các bước cài đặt simplex gồm dạng lỏng, pivot, khởi tạo và xử lý các trường hợp đặc biệt.

Từ khóa. Quy hoạch tuyến tính; thuật toán đơn hình; luồng đa hàng hóa.

1 Mô hình quy hoạch tuyến tính

Một bài toán quy hoạch tuyến tính tối ưu hóa một hàm tuyến tính dưới các ràng buộc cũng tuyến tính. Với các biến $x_1, \dots, x_n$, hàm mục tiêu có dạng

$$f(x_1, \dots, x_n) = \sum_{i=1}^n c_i x_i.$$

Mỗi ràng buộc là một đẳng thức hoặc bất đẳng thức tuyến tính, chẳng hạn

$$\sum_{i=1}^n a_i x_i \leq b, \quad \sum_{i=1}^n a_i x_i = b, \quad \sum_{i=1}^n a_i x_i \geq b.$$

Một nghiệm khả thi chỉ cần thỏa các ràng buộc; nghiệm tối ưu là nghiệm khả thi cho giá trị mục tiêu tốt nhất.

Ví dụ mở đầu của slide là bài tranh cử thị trưởng. Người tranh cử cần đủ phiếu ở ba khu vực: thành thị, ngoại ô và nông thôn. Bốn chính sách có tác động khác nhau lên ba nhóm cử tri; một chính sách có thể tăng phiếu ở nhóm này nhưng giảm ở nhóm khác. Nếu $x_1, \dots, x_4$ là số tiền chi cho bốn chính sách, mục tiêu là cực tiểu hóa

$$x_1 + x_2 + x_3 + x_4$$

với các ràng buộc

$$\begin{aligned}-2x_1 + 8x_2 + 10x_4 &\geq 50, \\ 5x_1 + 2x_2 &\geq 100, \\ 3x_1 - 5x_2 + 10x_3 - 2x_4 &\geq 25, \\ x_1, x_2, x_3, x_4 &\geq 0.\end{aligned}$$

Điểm quan trọng của ví dụ là các hệ số không nhất thiết dương: một lựa chọn có thể mang lợi ở ràng buộc này và gây thiệt ở ràng buộc khác, miễn là mọi quan hệ vẫn tuyến tính.

Hệ số của quy hoạch tuyến tính có thể là số thực bất kỳ, và nghiệm tối ưu cũng không bắt buộc là số nguyên. Nếu thêm yêu cầu nghiệm nguyên, bài toán trở thành quy hoạch nguyên, khó hơn nhiều và không còn thuộc phạm vi của simplex liên tục trong bài này.

2 Hai mô hình cơ bản

2.1 Phân bổ tài nguyên

Có m loại tài nguyên với giới hạn b_j , và n dự án. Nếu làm lượng x_i của dự án i , ta thu lợi $c_i x_i$ và tiêu tốn $a_{ij} x_i$ đơn vị tài nguyên j . Mô hình là

$$\begin{array}{ll}\text{cực đại} & \sum_{i=1}^n c_i x_i \\ \text{thỏa} & \sum_{i=1}^n a_{ij} x_i \leq b_j \quad (1 \leq j \leq m), \\ & x_i \geq 0 \quad (1 \leq i \leq n).\end{array}$$

Đây là khuôn mẫu cho các bài chọn sản lượng, chọn mức đầu tư hoặc phân phối nguồn lực khi mọi tiêu hao tỷ lệ tuyến tính với lượng được chọn.

2.2 Cung ứng vật tư

Ngược lại, có m loại nhu cầu cần đạt ít nhất b_j . Vật tư loại i cung cấp a_{ij} đơn vị cho nhu cầu j , với chi phí đơn vị c_i . Khi đó ta cực tiểu hóa

$$\sum_{i=1}^n c_i x_i$$

với các điều kiện

$$\sum_{i=1}^n a_{ij} x_i \geq b_j \quad (1 \leq j \leq m), \quad x_i \geq 0.$$

Slide dùng ví dụ khẩu phần dinh dưỡng: cơm, rau, thịt cung cấp đường, chất béo, protein và vitamin với tỷ lệ khác nhau; mục tiêu là đủ dinh dưỡng với chi phí nhỏ nhất.

3 Luồng đa hàng hóa

Ứng dụng quan trọng hơn trong bài là luồng đa hàng hóa. Khác với luồng mạng thông thường chỉ có một nguồn và một đích, ở đây có k cặp (s_i, t_i) , mỗi cặp cần chuyển d_i đơn vị hàng hóa. Với cạnh $e = (u_e, v_e)$

có dung lượng c_e , đặt $f_i(u_e, v_e)$ là lượng hàng hóa i đi qua cạnh đó. Mô hình khả thi là

$$\begin{aligned} & \text{cực tiểu} && 0 \\ & \text{thỏa} && \sum_{i=1}^k f_i(u_e, v_e) \leq c_e \quad (1 \leq e \leq E), \\ & && \sum_{p: v_p=v} f_i(u_p, v_p) = \sum_{q: u_q=v} f_i(u_q, v_q) \\ & && \quad (1 \leq i \leq k, v \neq s_i, t_i), \\ & && \sum_{e: u_e=s_i} f_i(u_e, v_e) = d_i \quad (1 \leq i \leq k), \\ & && f_i(u_e, v_e) \geq 0. \end{aligned}$$

Hàm mục tiêu bằng 0 nghĩa là chỉ cần tìm một nghiệm khả thi. Nếu mỗi cạnh có chi phí đơn vị Cost_e , ta thay mục tiêu bằng

$$\text{cực tiểu} \quad \sum_{e=1}^E \text{Cost}_e \sum_{i=1}^k f_i(u_e, v_e).$$

Slide nhấn mạnh sự khác biệt với luồng mạng cổ điển: mô hình này cho phép nhiều loại hàng cùng dùng chung dung lượng cạnh. Chính vì vậy nó biểu đạt được nhiều bài vận tải và thiết kế mạng, nhưng nghiệm tổng quát là nghiệm thực, không tự động nguyên như luồng một hàng hóa.

4 Đưa về dạng chuẩn

Dạng chuẩn được dùng trong phần cài đặt là:

$$\begin{aligned} & \text{cực đại} && \sum_{j=1}^n c_j x_j \\ & \text{thỏa} && \sum_{j=1}^n a_{ij} x_j \leq b_i \quad (1 \leq i \leq m), \\ & && x_j \geq 0 \quad (1 \leq j \leq n). \end{aligned}$$

Các dạng khác được biến đổi bằng những quy tắc quen thuộc:

- cực tiểu hóa f tương đương cực đại hóa $-f$;
- ràng buộc $q(x) = b$ được thay bằng $q(x) \leq b$ và $q(x) \geq b$;
- ràng buộc $\sum a_{ij} x_j \geq b$ được nhân -1 ;
- biến tự do x được thay bằng hiệu $x^+ - x^-$, với $x^+, x^- \geq 0$.

Để chạy đơn hình, ta thêm biến dư:

$$x_{n+i} = b_i - \sum_{j=1}^n a_{ij} x_j \quad (1 \leq i \leq m).$$

Các biến x_{n+i} ban đầu là biến cơ sở, còn $x_1, \dots, x_n$ là biến phi cơ sở. Khi mọi $b_i \geq 0$, đặt biến phi cơ sở bằng 0 đã cho một nghiệm khả thi.

5 Pivot và chương trình chính

Một trạng thái simplex được xem như dạng lỏng: mỗi biến cơ sở được biểu diễn theo các biến phi cơ sở, còn hàm mục tiêu cũng được viết theo các biến phi cơ sở. Nếu tồn tại một biến phi cơ sở x_e có hệ số

dương trong hàm mục tiêu, ta muốn tăng x_e . Để giữ các biến cơ sở không âm, chọn biến ra x_l theo phép so tỷ lệ:

$$\frac{b_l}{A_{le}} = \min_{i \in B, A_{ie} > 0} \frac{b_i}{A_{ie}}.$$

Sau đó pivot đổi vai trò x_e và x_l , đồng thời thay phương trình của x_e vào mọi phương trình còn lại và vào hàm mục tiêu.

Nếu không có hàng nào thỏa $A_{ie} > 0$, có thể tăng x_e vô hạn mà vẫn giữ khả thi; bài toán không bị chặn. Nếu mọi hệ số mục tiêu của biến phi cơ sở đều không dương, nghiệm hiện tại là tối ưu, vì tăng bất kỳ biến phi cơ sở nào cũng không làm mục tiêu lớn hơn.

Slide có mã giả cho pivot, opt và init; ở đây chỉ tóm tắt theo ngoại lệ dành cho mẫu mã dài. Phần cốt lõi của opt là lặp: chọn biến vào có hệ số mục tiêu dương, chọn biến ra bằng phép so tỷ lệ, rồi pivot. Để tránh vòng lặp do suy biến, có thể dùng quy tắc Bland: khi có nhiều lựa chọn, ưu tiên chỉ số nhỏ hơn.

6 Khởi tạo và chi tiết cài đặt

Nếu ban đầu có $b_i < 0$, dạng lỏng chưa cho nghiệm khả thi trực tiếp. Cách khởi tạo trong bài là dựng một bài toán phụ bằng cách thêm biến x_0 :

$$x_i = b_i - \sum_{j \in N} A_{ij}x_j + x_0,$$

và cực đại hóa $-x_0$. Sau một pivot đặc biệt với hàng có b_i nhỏ nhất, ta tối ưu bài phụ. Nếu giá trị tối ưu của bài phụ nhỏ hơn 0, bài gốc vô nghiệm; nếu bằng 0, bỏ x_0 và tiếp tục tối ưu bài gốc.

Khi cài bằng số thực, cần đặt ngưỡng ε cho so sánh với 0, nhất là khi kiểm tra $c_e > 0$, $A_{ie} > 0$ và các tỷ lệ gần bằng nhau. Simplex không có cận đa thức tổng quát cho mọi dữ liệu, nhưng kinh nghiệm của tác giả là với dữ liệu thi lập trình, một cài đặt khoảng vài trăm dòng thường đủ nhanh. Các thử nghiệm trong bài gốc so sánh chương trình với MATLAB và LINDO để minh họa rằng tốc độ thực tế của simplex rất đáng tin cậy.

7 Kết luận

Quy hoạch tuyến tính không thay thế các mô hình rời rạc kinh điển như luồng mạng hay ghép cặp, nhưng là một công cụ mạnh khi bài toán thật sự tuyến tính và nghiệm liên tục được chấp nhận. Khi gặp mục tiêu tuyến tính, ràng buộc tuyến tính và các biến không âm, việc thử dựng mô hình LP có thể mở ra một hướng giải hoàn toàn khác. Với simplex, rào cản cài đặt thấp hơn vẻ ngoài của nó: phần khó nhất thường là nhận ra mô hình và chuẩn hóa dữ liệu đầu vào đúng cách.